# Sagar Fab International Company

*Full Stack Developer Assignment - AI Tools Usage Report*

---

## Candidate Evaluation Report

Role: Full Stack Developer (Ruby on Rails + Flutter)

Date: July 2, 2026

### 1. AI Tools Utilized

During the development of this assignment, the Google DeepMind agentic coding assistant "Antigravity" was utilized as a primary development partner. The tool was operated in an interactive, pair-programming fashion, where the developer directed architectural goals, audited code blocks, and refined layouts, while the AI generated templates, model parameters, and database schemas.

### 2. How the AI was Leveraged

Rather than relying on blind generation, the AI was directed through progressive, iterative stages of feature scoping:

* **Phase 1 - Backend Scaffolding:** Setting up the SQLite3 database migrations, configuring Rails routes, and generating validation rules.
* **Phase 2 - PDF Metadata Extraction:** Utilizing Ruby gems to read metadata parameters inside uploaded PDF files to automatically set title and author defaults.
* **Phase 3 - Flutter Integration:** Developing custom UI widgets such as the classical wooden Bookshelf View, the 3D book cover card, and horizontal scroll areas.
* **Phase 4 - Testing & Quality Assurance:** Writing unit tests for Rails controllers and widget tests for Flutter widgets.

### 3. Key AI-Assisted Components

* **Bookshelf UI:** Math logic for dividing the list of books into shelves depending on device screen width, and drawing visual mahogany boards underneath.
* **Dynamic Covers:** Generating random colors on the backend and delivering hex pairs to draw glossy 3D covers dynamically on the client.
* **Search Debouncing:** Timer debounce structures in the library search bar to bundle queries and prevent server load.
* **Page Memory & Caching:** Reading and writing last read page indices to SharedPreferences on scroll events.
* **Text Matching Highlight:** The case-insensitive search highlight algorithm that matches keywords and injects styled TextSpans.

### 4. Developer Modifications, Corrections & Auditing

A strong emphasis was placed on auditing and modifying the code to maintain type safety and handle platform-specific bugs:

* **Active Storage Gotcha Corrected:** The initial AI plan proposed reading file headers before validation using

"file.open". The developer flagged that Active Storage files are not written to disk until after database save transaction commits. The developer rewrote the parser to check temporary files in "attachment_changes" directly before validation, preventing active storage file-not-found exceptions.

* **Visual Overflow Bugs Solved:** The developer audited layout boundaries on the emulator and directed two main fixes: (a) wrapping metadata tags in a Wrap widget rather than a Row in the List View, and (b) adding checks to hide covers text on small previews (height < 100) to resolve vertical constraints overflows.
* **SDK Deprecations Upgraded:** The developer updated outdated Material theme parameter types generated by the AI to match current Flutter 3 guidelines (e.g. converting DialogTheme to DialogThemeData).

## 5. Impact on Architecture, Testing, and Product Decisions

* **Architecture:** The codebase is split cleanly into a standalone backend and frontend. The dynamic color cover generation avoids slow and unstable server-side PDF cover page image rendering libraries.
* **Testing:** The Rails integration test suite ensures multipart uploads, downloads, and searches work correctly, while Flutter widget tests cover visual smoke checks.
* **Product Decisions:** The addition of the "Continue Reading" horizontal list and custom cover image uploads significantly elevates the application to feel like a premium, production-grade library.